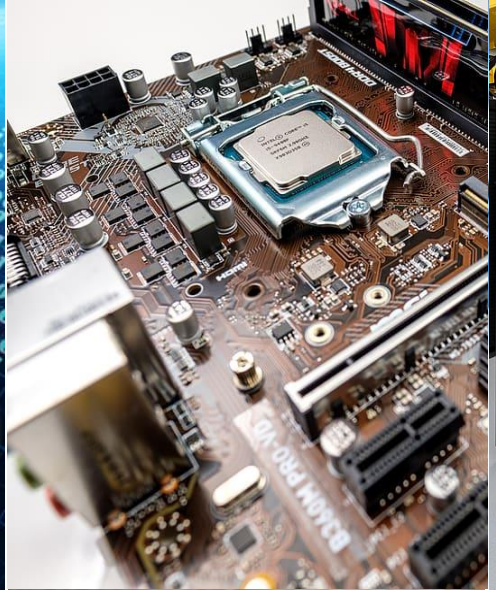
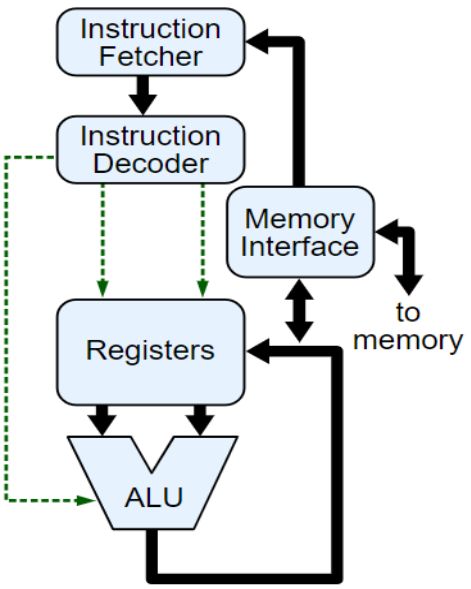



summit
IBM


Instruction Fetcher
Instruction Decoder
Memory Interface
to memory
Registers
ALU

Recap

- We covered (in length) how integers are represented and used by computers.
- Here, we will cover the other primitives used by most programming languages.

Representing characters

- In early days (1950's), computers worked with different "byte" length (6-, 7-, or 8-bits) depending on the character set size.
- In 1963, the American Standard Association proposed the 7-bit American Standard Code for Information Interchange (ASCII) encoding.
- IBM ® System/360 (1964) used 8-bits bytes, changing the 6-bits standard as character processing became more important than digit processing.

The 7-bit American Standard Code for Information Interchange (ASCII) encoding

	010		011		100		101		110		111	
	ASCII value	Char-acter	ASCII value	Char-acter	ASCII value	Char-acter	ASCII value	Char-acter	ASCII value	Char-acter	ASCII value	Char-acter
0000	32	space	48	0	64	@	80	P	96	`	112	p
0001	33	!	49	1	65	A	81	Q	97	a	113	q
0010	34	"	50	2	66	B	82	R	98	b	114	r
0011	35	#	51	3	67	C	83	S	99	c	115	s
0100	36	$	52	4	68	D	84	T	100	d	116	t
0101	37	%	53	5	69	E	85	U	101	e	117	u
0110	38	&	54	6	70	F	86	V	102	f	118	v
0111	39	'	55	7	71	G	87	W	103	g	119	w
1000	40	(	56	8	72	H	88	X	104	h	120	x
1001	41	)	57	9	73	I	89	Y	105	i	121	y
1010	42	*	58	:	74	J	90	Z	106	j	122	z
1011	43	+	59	;	75	K	91	[	107	k	123	{
1100	44	,	60	<	76	L	92	\	108	l	124	|
1101	45	-	61	=	77	M	93	]	109	m	125	}
1110	46	.	62	>	78	N	94	^	110	n	126	~
1111	47	/	63	?	79	O	95	_	111	o	127	DEL

- Say that x encodes a letter in ASCII.
  - How'd you make sure it's capitalized? $x \,\&= \sim 0b100000$
  - How'd you switch its case? $x \,\hat{}= 0b100000$
- Say that y encodes a digit in ASCII.
  - How'd you get its value? $y - '0' = y - 48$

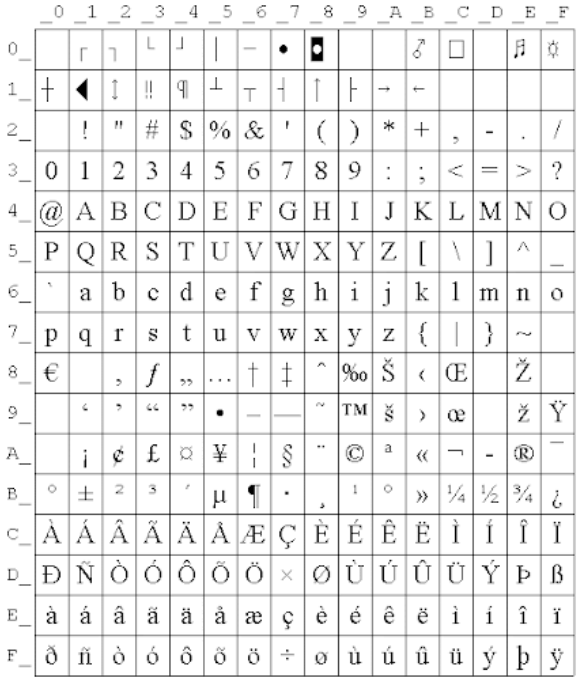
What about ASCII values 0-31?

- These are *control* (non-printable) characters.

Dec	Hex	Name	Char	Ctrl-char
0	0	Null	NUL	CTRL-@
1	1	Start of heading	SOH	CTRL-A
2	2	Start of text	STX	CTRL-B
3	3	End of text	ETX	CTRL-C
4	4	End of xmit	EOT	CTRL-D
5	5	Enquiry	ENQ	CTRL-E
6	6	Acknowledge	ACK	CTRL-F
7	7	Bell	BEL	CTRL-G
8	8	Backspace	BS	CTRL-H
9	9	Horizontal tab	HT	CTRL-I
10	0A	Line feed	LF	CTRL-J
11	0B	Vertical tab	VT	CTRL-K
12	0C	Form feed	FF	CTRL-L
13	0D	Carriage feed	CR	CTRL-M
14	0E	Shift out	SO	CTRL-N
15	0F	Shift in	SI	CTRL-O
16	10	Data line escape	DLE	CTRL-P
17	11	Device control 1	DC1	CTRL-Q
18	12	Device control 2	DC2	CTRL-R
19	13	Device control 3	DC3	CTRL-S
20	14	Device control 4	DC4	CTRL-T
21	15	Neg acknowledge	NAK	CTRL-U
22	16	Synchronous idle	SYN	CTRL-V
23	17	End of xmit block	ETB	CTRL-W
24	18	Cancel	CAN	CTRL-X
25	19	End of medium	EM	CTRL-Y
26	1A	Substitute	SUB	CTRL-Z
27	1B	Escape	ESC	CTRL-[
28	1C	File separator	FS	CTRL-\
29	1D	Group separator	GS	CTRL-]
30	1E	Record separator	RS	CTRL-^
31	1F	Unit separator	US	CTRL-_

Hey, where's the £ sign?

- ASCII has no support for many common characters.
- The UK ISO 646 variant (1967) replaced # with £.
- Extended ASCII uses 8-bits to encode more characters within a byte.



	_0	_1	_2	_3	_4	_5	_6	_7	_8	_9	_A	_B	_C	_D	_E	_F
0_		┌	┐	└	┘	│	–	•	◘			♂	□		♫	☼
1_	┼	◄	↕	‼	¶	┴	┬	┤	↑	├	→	←				
2_		!	"	#	$	%	&	'	(	)	*	+	,	-	.	/
3_	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4_	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5_	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6_	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7_	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	
8_	€		‚	ƒ	„	…	†	‡	ˆ	‰	Š	‹	Œ		Ž	
9_		‘	’	“	”	•	–	—	˜	™	š	›	œ		ž	Ÿ
A_		¡	¢	£	¤	¥	¦	§	¨	©	ª	«	¬	-	®	¯
B_	°	±	²	³	´	µ	¶	·	¸	¹	º	»	¼	½	¾	¿
C_	À	Á	Â	Ã	Ä	Å	Æ	Ç	È	É	Ê	Ë	Ì	Í	Î	Ï
D_	Ð	Ñ	Ò	Ó	Ô	Õ	Ö	×	Ø	Ù	Ú	Û	Ü	Ý	Þ	ß
E_	à	á	â	ã	ä	å	æ	ç	è	é	ê	ë	ì	í	î	ï
F_	ð	ñ	ò	ó	ô	õ	ö	÷	ø	ù	ú	û	ü	ý	þ	ÿ

ISO Latin-1

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0				0	@	P	`	p				°	Ŕ	Đ	ŕ	đ
1			!	1	A	Q	a	q			Ą	ą	Á	Ń	á	ń
2			"	2	B	R	b	r			˘	˛	Â	Ň	â	ň
3			#	3	C	S	c	s			Ł	ł	Ă	Ó	ă	ó
4			$	4	D	T	d	t			¤	´	Ä	Ô	ä	ô
5			%	5	E	U	e	u			Ľ	ľ	Ĺ	Ő	Í	ő
6			&	6	F	V	f	v			Ś	ś	Ć	Ö	ć	ö
7			'	7	G	W	g	w			§	ˇ	Ç	×	ç	÷
8			(	8	H	X	h	x			¨	¸	Č	Ř	č	ř
9			)	9	I	Y	i	y			Š	š	É	Ů	é	ů
A			*	:	J	Z	j	z			Ş	ş	Ę	Ú	ę	ú
B			+	;	K	[	k	{			Ť	ť	Ë	Ű	ë	ű
C			,	<	L	\	l	|		Ź	ź	Ě	Ü	ě	ü
D			-	=	M	]	m	}			-	˝	Í	Ý	í	ý
E			.	>	N	^	n	~			Ž	ž	Î	Ţ	î	ţ
F			/	?	O	_	o				Ż	ż	Ď	ß	ď	˙

ISO Latin-2

A global solution -- Unicode

- Uses up to 4-bytes per character. (0-0x10FFFF).
- Currently includes ≈145k characters.
  - All the world languages.
  - Symbols like emoji.
  - Compatible with EASCII Latin-1.
- Has multiple encodings.
  - The simplest of which is UTF-32, which uses exactly 32-bits per character.
  - Brings the issue of endianness (resolved by a special character that precedes the text).

UTF8 -- An efficient, backward compatible, encoding

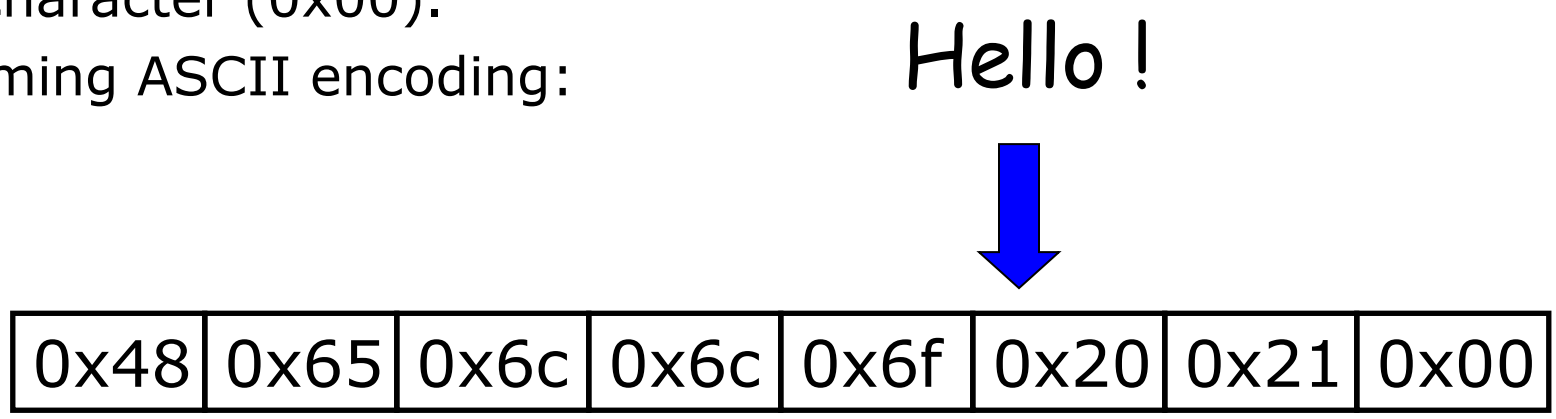
- A *variable-length* encoding (Thompson and Pike, 1992).
- ASCII characters are supported as 1-byte characters.
  - If a character's encoding starts with 0 (i.e., 0xxxxxxx) then it is the ASCII character xxxxxxx.
- If the encoding starts with 110, it must follow the form (110yyyyy 10yyyyyy) and encode the Unicode character 0byyyyyyyyyyy.
  - Covers almost all Latin-script alphabets in addition to Greek, Cyrillic, Coptic, Armenian, Hebrew, Arabic.
- If the encoding starts with 1110, it will follow 1110zzzz 10zzzzzz 10zzzzzz.
  - Gives 16-bits, enough for the Basic Multilingual Plane.
- The last option is 11110aaa 10aaaaaa 10aaaaaa 10aaaaaa, encoding the remaining characters.

What did we learn about character encodings?

Lecture 5: Character Encoding

From characters to strings

- Python “str” and Java “String” classes are internally complex and employ Unicode characters.
- The C representation of a string is much simpler:
  - Terminated with a null character (0x00).
  - Assuming ASCII encoding:



Representing real numbers

- Back in week 1, we discussed the "point" notation.
  - $27.3_{10} = 2 \cdot 10^1 + 7 \cdot 10^0 + 3 \cdot 10^{-1}$.
  - $10.11_2 = 1 \cdot 2^1 + 0 \cdot 2^0 + 1 \cdot 2^{-1} + 1 \cdot 2^{-2} = 2.75_{10}$.
- But how do computers know where the dot is?
- And how do we represent negative real numbers?

The *fixed*-point representation

- In some cases, we know the *scale* (number of digits before the point) of the number.
  - For example, prices are often shown with two fractional digits: £86.73.
  - In such cases, we can store the integer 8673 to represent 86.73.
  - Negative number representation follows seamlessly, e.g., representing -86.73 as the 2's comp. representation of -8673.

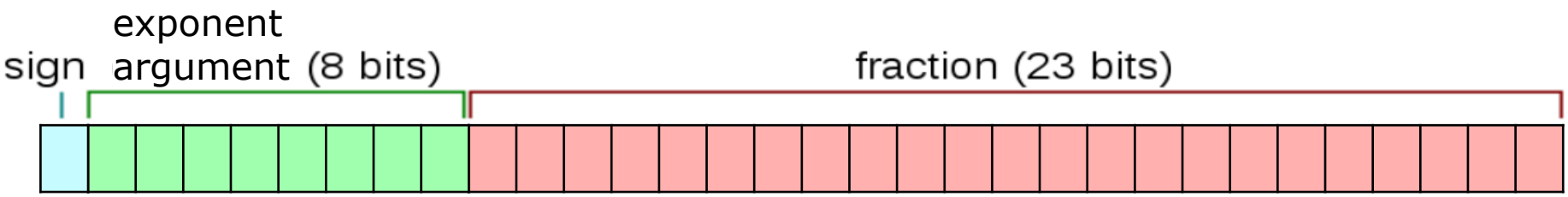
Floating point representation

- In cases where we don't know the scale, it must be represented as part of the number.

- $1101.11 = 1.10111 \cdot 10^{3}$.

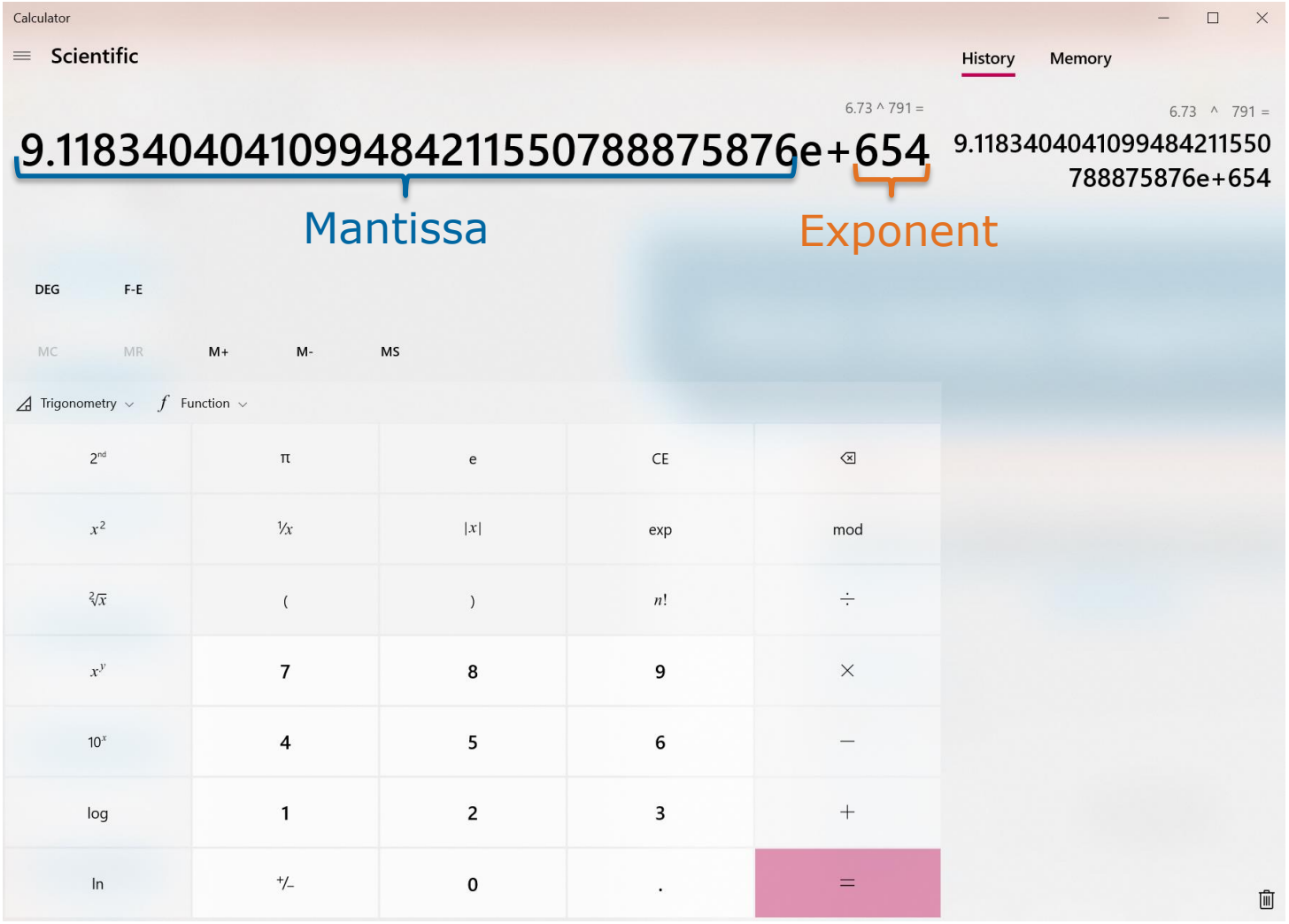
Exponent

Mantissa (fraction)



IEEE 754 format for single-precision floats

How many digits?



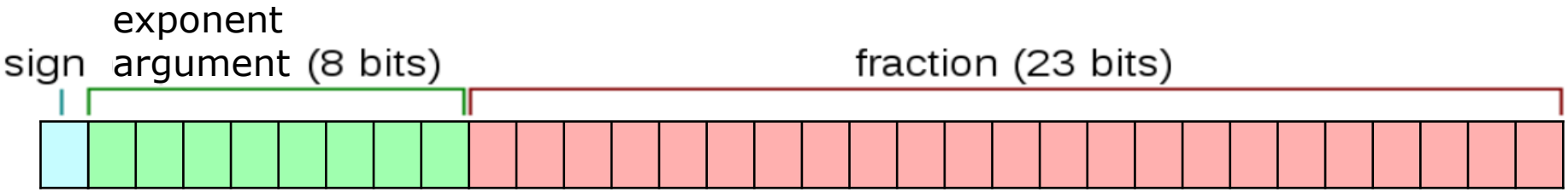
IEEE 754 single precision

- Only a fixed amount of mantissa bits are encoded (i.e., there's an accuracy loss).
- The exponent is not encoded directly but with a *bias*.
  - The Exp. argument (EA) is in the range $\{1, \ldots, 254\}$ for *normal numbers*.
  - True exponent = EA – B, where B=127_{10} is the bias.

$$\textbf{Value} = (-1)^{sign} \cdot 2^{EA-B} \cdot 1.\textbf{fraction} = (-1)^{sign} \cdot 2^{EA-B} \cdot \left(1 + \sum_{0}^{22} x_i \cdot 2^{i-23}\right)$$

  - E.g., 11000001101000000000000000000000

$$\text{represents } -2^{131-127} \cdot 1.01_2 = -16 \cdot 1.25 = -20_{10}.$$



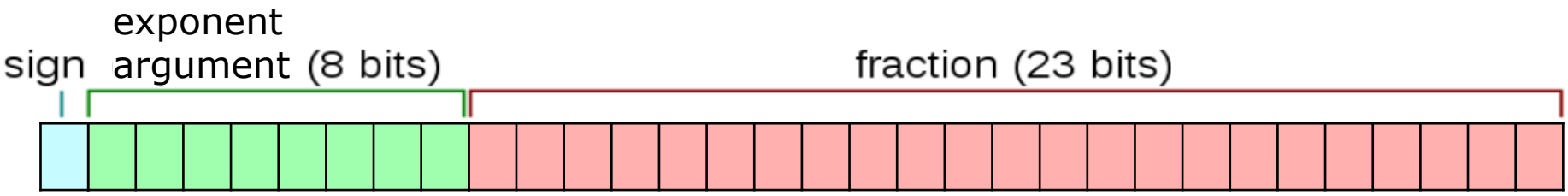
IEEE 754 single precision

- The exponent is not encoded directly but with a bias.
  - The Exp. argument (EA) is in the range $\{1, \ldots, 254\}$ for *normal numbers*.
  - True exponent = EA – B, where B=127_{10} is the bias.

⇒ The true exponent is in the range $[1 - 127, 254 - 127] = [-126, 127]$.

⇒ The largest representable number is $(2 - 2^{-23}) \cdot 2^{127} \approx 2^{128}$.

⇒ The smallest representable ***normal*** number is 2^{-126}.

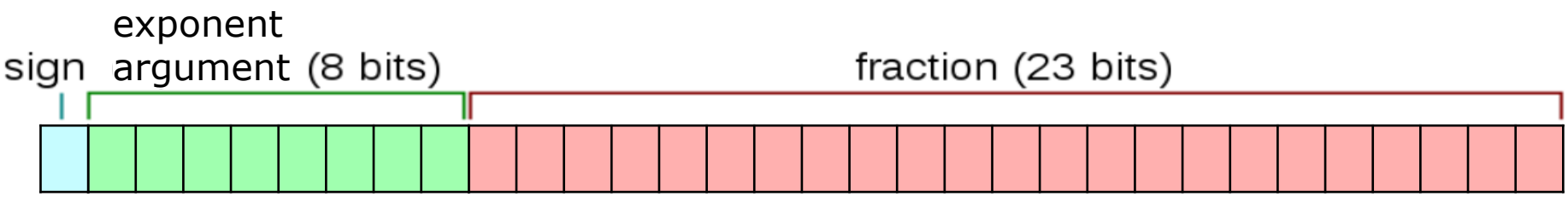


Subnormal numbers (single prec.)

- Represented with EA=0.
- Represent $(0.\text{fraction} \cdot 2^{-126})$. For example:
  - $10000000001000000000000000000000$ represents $-2^{-126} \cdot 0.01_2 = -2^{-128}$.
  - $00000000011111111111111111111111$ represents

    $$2^{-126} \cdot 0.1\ldots1_2 = 2^{-126} \cdot (1 - 2^{-23}).$$

  - $00000000000000000000000000000001$ is $+2^{-126} \cdot 0.0\ldots01_2 = 2^{-149}$.
  - $00000000000000000000000000000000$ is +0 (and there's also -0!).



Special numbers

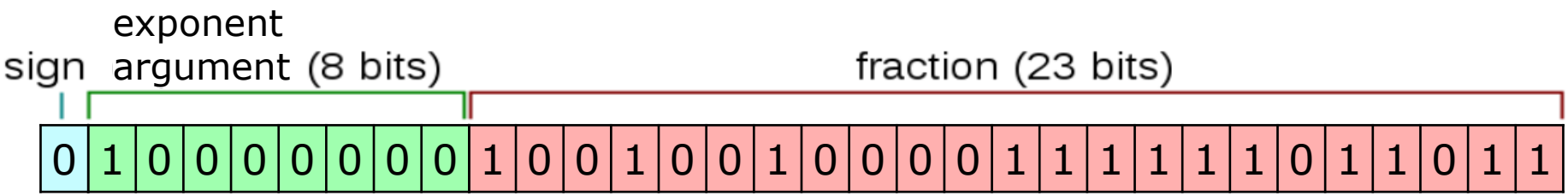
- What happens if $EA = 255$?
  - A mantissa of 0 means ∞.
    - There is also $-\infty$.
  - A non-zero mantissa indicated Not a Number (NaN).
- Supports arithmetic operations!

Operation	Result
n ÷ ±Infinity	0
±Infinity × ±Infinity	±Infinity
±nonZero ÷ ±0	±Infinity
±finite × ±Infinity	±Infinity
Infinity + Infinity Infinity – -Infinity	+Infinity
-Infinity – Infinity -Infinity + – Infinity	– Infinity
±0 ÷ ±0	NaN
±Infinity ÷ ±Infinity	NaN
±Infinity × 0	NaN
NaN == NaN	False

Exercise: represent π as float

- Let us represent (an approximation of) π using single prec.

$$\pi \approx 3.14159\ 26535\ 89793_{10}$$
$$\approx 11.001001000011111101101010100010001000010110100010111011110\ldots_2$$
$$\approx 10^1 \cdot 1.10010010000111111011011$$
$$= +10^{128-127} \cdot 1.10010010000111111011011$$



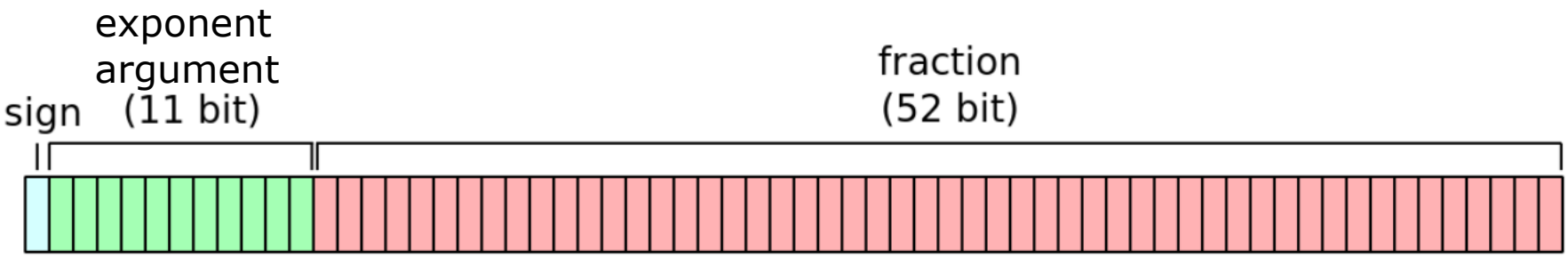
- How accurate is this?
  - The actual number represented is: 3.141592741012573242187 5.
  - The decimal rep. of π starts with: 3.141592653589793238462 6.
  - A relative error of $\approx 3 \cdot 10^{-8} < 2^{-24}$.

IEEE 754 double precision

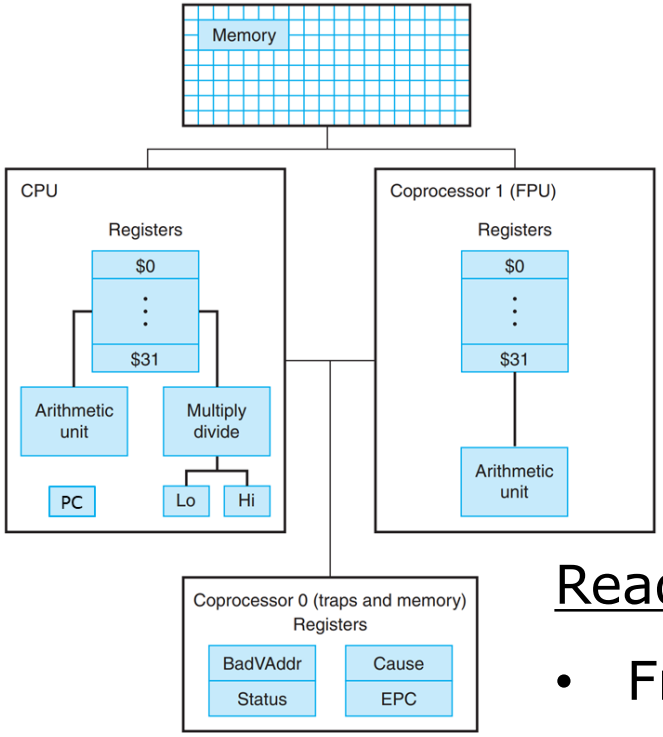
- Similar, but with 64 bits.
- Uses a bias of $B = 1023$.
- Normal numbers have $EA \in [1,2046]$.
- $EA = 0$ is used for subnormals.
- $EA = 2047$ is for NaN and $\pm\infty$.

There are also:

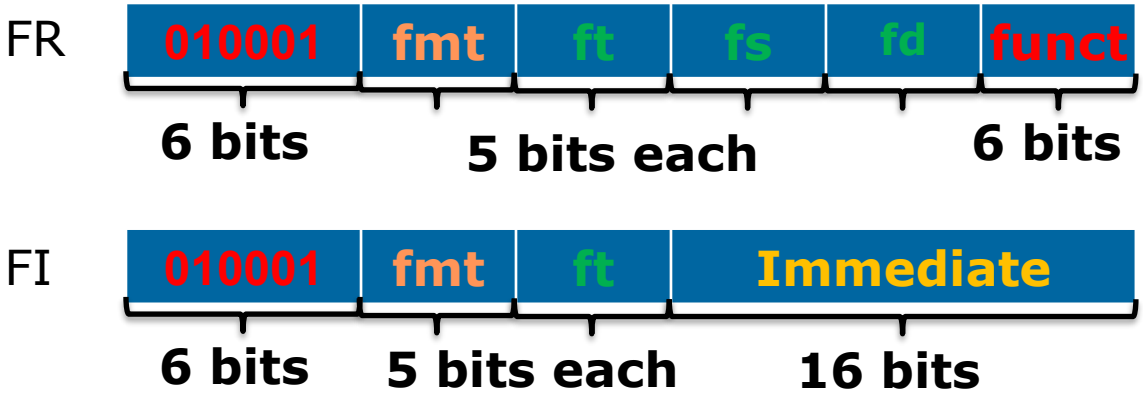
- Quad precision (128 bits)
- Half precision (16 bits)
- FP8/FP6/FP4



The MIPS floating point co-processor



Format



Reading to FP registers:

- From memory:

```
lwc1 $f1, 100($s2)  Load word to cop. 1.
swc1 $f1, 100($s2)  Store word from cop. 1.
```

- From Registers:

```
mtc1 $t2, $f2  Load word to cop. 1.
```

MIPS Arithmetic Floating Point Instructions

Instruction	Description
`add.s $f2, $f4, $f6`	FP add (single precision)
`sub.s $f2, $f4, $f6`	FP subtract (single precision)
`mul.s $f2, $f4, $f6`	FP multiple (single precision)
`div.s $f2, $f4, $f6`	FP divide (single precision)
`add.d $f2, $f4, $f6`	FP add (double precision)
`sub.d $f2, $f4, $f6`	FP subtract (double precision)
`mul.d $f2, $f4, $f6`	FP multiple (double precision)
`div.d $f2, $f4, $f6`	FP divide (double precision)

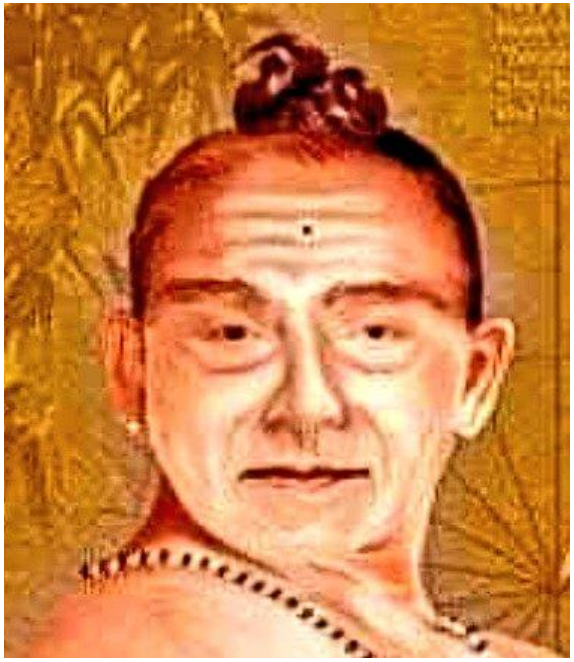
Floating point registers go from $f0 to $f31.

Double precision is 64-bit and uses 2 adjacent registers (hence only $f0, $f2, $f4, … are allowed)

Converting floats and ints

- Use the `cvt` (convert) instructions.
  - `cvt.s.w` (convert word to single-prec. float)
  - `cvt.d.w` (convert word to double-prec. float)
  - `cvt.d.s / cvt.s.d` (convert single to double or vice versa)
  - `cvt.w.s / cvt.w.d` (convert to word)
  - `cvt.l.* / cvt.*.l` (convert to/from long – ***MIPS64***)

Example: calculating π like in the old times



Madhava of Sangamagrama 1340-1425

$$\pi = 4 \sum_{n=0}^{\infty} \frac{(-1)^n}{2n+1}$$

$$= 4 \cdot \arctan(1)$$

Example: calculating π like in the old times

```
li $8, 0 #iteration number, i.e., n ≡ $8.
li $15, 10000 #number of summands

mtc1 $0, $f0 # $f0 will be our sum

li $9, 1 # $9 is the constant 1
mtc1 $9, $f1
cvt.s.w $f1, $f1 # $f1 is the constant 1

j loop #after finishing, $f0 is approx. π/4
end:
li $13, 4
mtc1 $13, $f4
cvt.s.w $f4, $f4 # $f4 is the constant 4

mul.s $f0, $f0, $f4

li $v0, 2          # syscall 2: print float
mov.s $f12, $f0    # Move contents of register $f0 to register $f12
syscall
```

$$\pi = 4 \sum_{n=0}^{\infty} \frac{(-1)^n}{2n+1}$$

Example: calculating π like in the old times

```
loop:   sll $12, $8, 1
        addi $12, $12, 1 # $12 = (2*$8 + 1)

        mtc1 $12, $f2
        cvt.s.w $f2, $f2 # $f2 = (2*$8 + 1)

        div.s $f2, $f1, $f2 # $f2 = 1/(2*$8 + 1)

        andi $11, $8, 1  # $11 = ($8 % 2)
        sll $11, $11, 1  # $11 = 2*($8 % 2)
        sub $11,$9, $11 # $11 = (-1)^$8
        mtc1 $11, $f3
        cvt.s.w $f3, $f3 # $f3 = (-1)^$8

        mul.s $f2, $f2, $f3 # $f2 = (-1) ^ $8 /(2*$8 + 1)
        add.s $f0, $f0, $f2

        addi $8, $8, 1
        bne $8, $15, loop
        j end
```

$$\pi = 4 \sum_{n=0}^{\infty} \frac{(-1)^n}{2n+1}$$

Lecture Summary

- We discussed representation of characters, strings, and real numbers.
- Studied the IEEE 754 floating point format.
  - We have looked at normalized format (0 < E < 255).
  - Denormalized format (E = 0).
  - Special values NaN: +Inf, -Inf (encoded with E=255).
- We introduced the FPU (Floating Point Unit) of the MIPS CPU and calculated the value of π.